



UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA

Scuola di Scienze

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di Laurea Magistrale in Informatica

Metodi del Calcolo Scientifico: Assignment 2

Report prodotto da:

Broglia Matteo – 899562

Caputo Lorenzo – 894528

Giuggioli Daniel – 894415

Anno Accademico 2025 - 2026

Indice

1	Struttura del Progetto	1
1.1	Organizzazione dei File	1
1.2	Linguaggio e Librerie	2
1.3	Utilizzo ed Esecuzione	2
1.3.1	Applicazione Grafica	2
1.4	Approfondimento: scipy.fftpack	3
1.4.1	Complessità Computazionale	3
1.4.2	Firma del Metodo e Parametri	4
1.4.3	Integrazione nel Codice Sviluppato	5
2	Metodi DCT e Compressione	7
2.1	Trasformata Coseno Discreta (DCT)	7
2.1.1	Definizione Matematica	7
2.1.2	DCT 1D: Implementazione Manuale	8
2.1.3	DCT 2D: Implementazione Manuale	8
2.1.4	DCT 2D: Implementazione Veloce	9
2.2	Trasformata Inversa (IDCT)	10
2.3	Algoritmo di Compressione	10
2.3.1	Analisi del Criterio di Sogliatura	11
3	Risultati	13
3.1	Test sulle Matrici	13
3.1.1	Test Matrice 1D	13
3.1.2	Test Matrice 2D	14
3.2	Benchmark	15
3.2.1	Configurazione del Test	15
3.2.2	Risultati	16
3.2.3	Conclusioni	16
3.3	Compressione su Immagini	17
3.3.1	Parametri e Configurazione	17
3.3.2	Risultati	17
3.3.3	Analisi dei Risultati	18
3.3.4	Osservazioni	19
3.3.5	Conclusioni Finali	21
3.4	Dimensione del File	21
3.4.1	Problema Riscontrato	21
3.4.2	Causa	21
3.4.3	Soluzione Adottata	22

3.4.4	Validazione della Soluzione	23
4	Conclusioni	25

Capitolo 1

Struttura del Progetto

Questo progetto implementa un'applicazione Python per la compressione di immagini utilizzando la Trasformata Coseno Discreta (DCT - Discrete Cosine Transform)

Il codice è reperibile tramite GitHub al seguente indirizzo

<https://github.com/mbroglio/ImageCompression>

1.1 Organizzazione dei File

Il codice sorgente è stato suddiviso in moduli per garantire una netta separazione delle responsabilità (matematica, compressione, interfaccia e validazione).

Moduli Core

- `dct.py` (Modulo Matematico):
 - `compute_D(N)`: Calcola la matrice DCT ortonormale $N \times N$ (DCT-II con scaling ortogonale standard).
 - `custom_dct1(f)`: DCT 1D tramite moltiplicazione matriciale ($\mathcal{O}(N^2)$).
 - `custom_dct2(f)`: DCT 2D tramite applicazione iterativa della trasformata 1D sulle dimensioni ($\mathcal{O}(N^3)$).
 - `fast_dct2(f)`: DCT 2D ottimizzata tramite `scipy.fftpack` ($\mathcal{O}(N^2 \log N)$).
 - `fast_idct2(c)`: Trasformata inversa veloce per la ricostruzione del segnale.
- `compressor.py` (Modulo di Compressione):
 - `compress_image(path, F, d)`: Implementa l'algoritmo di compressione. Divide l'immagine in blocchi $F \times F$, applica la DCT veloce, azzeri i coefficienti in posizione (k, l) tali che $k + l \geq d$, applica la IDCT e ricompone l'immagine.

Interfaccia e Risorse

- `app.py`: Applicazione desktop basata su `customtkinter`. Gestisce la selezione dei file, l'input dei parametri e il rendering a schermo dei risultati.
- `images/`: Directory contenente le immagini di test (es. BMP originali).

Testing, Validazione e Benchmark

- `test_matrices.py`: Suite di unit test per la validazione delle matrici contro i valori attesi (come da specifica).
- `benchmark.py`: Script che confronta le prestazioni temporali tra la DCT custom ($\mathcal{O}(N^3)$) e la DCT veloce ($\mathcal{O}(N^2 \log N)$) al variare della dimensione N . Genera il grafico `performance_comparison.png` e salva i dati in `benchmark_results.csv`.
- `validate_compression.py`: Script per l'analisi empirica della compressione su immagini reali:
 - `calculate_psnr(original, compressed)`: Calcola il Peak Signal-to-Noise Ratio (PSNR) come $10 \log_{10}(255^2/\text{MSE})$.
 - `run_compression_tests()`: Esegue test automatizzati sulle immagini di riferimento (`gradient.bmp`, `bridge.bmp`, `shoe.bmp`), misurando i tempi di esecuzione e salvando i log in `compression_results.csv`.
 - `test_fixed_F_varying_d()`: Analisi parametrica (fissato $F = 8$, $d \in \{2, 4, 8\}$).
 - `test_fixed_d_varying_F()`: Analisi parametrica (fissato $d = 3$, $F \in \{4, 10, 16\}$).

1.2 Linguaggio e Librerie

Il progetto è interamente sviluppato in Python e sfrutta le seguenti librerie open-source per il calcolo scientifico e le interfacce grafiche:

- **NumPy**: Gestione degli array multidimensionali e algebra lineare (calcolo della matrice $\mathbf{D}$ e prodotti matrice-vettore).
- **SciPy**: Implementazione efficiente della DCT/IDCT basata su FFT.
- **Pillow**: I/O delle immagini, conversione in scala di grigi, salvataggio su disco in formato BMP e codifica in-memory in formato PNG lossless per la stima della dimensione compressa.
- **customtkinter**: Sviluppo della GUI moderna (basata sul framework Tkinter).
- **Matplotlib**: Generazione dei grafici analitici per i benchmark.

1.3 Utilizzo ed Esecuzione

1.3.1 Applicazione Grafica

L'interfaccia desktop, principale punto d'accesso per l'utente, si avvia tramite terminale:

```
python app.py
```

Tramite la GUI è possibile regolare interattivamente due parametri fondamentali:

- **Macro-block Size (F)**: Dimensione dei quadrati $F \times F$ di analisi DCT. Un valore di $F = 8$ corrisponde allo standard JPEG classico. Range: $1 \leq F \leq \min\{\text{height}, \text{width}\}$. Range tipico: $4 \leq F \leq 16$

- **Cutoff Threshold (d):** Soglia di taglio delle frequenze spaziali. I coefficienti con indice (k, l) tali che $k + l \geq d$ vengono scartati (azzerati). Range ammesso: $0 \leq d \leq 2F - 2$.

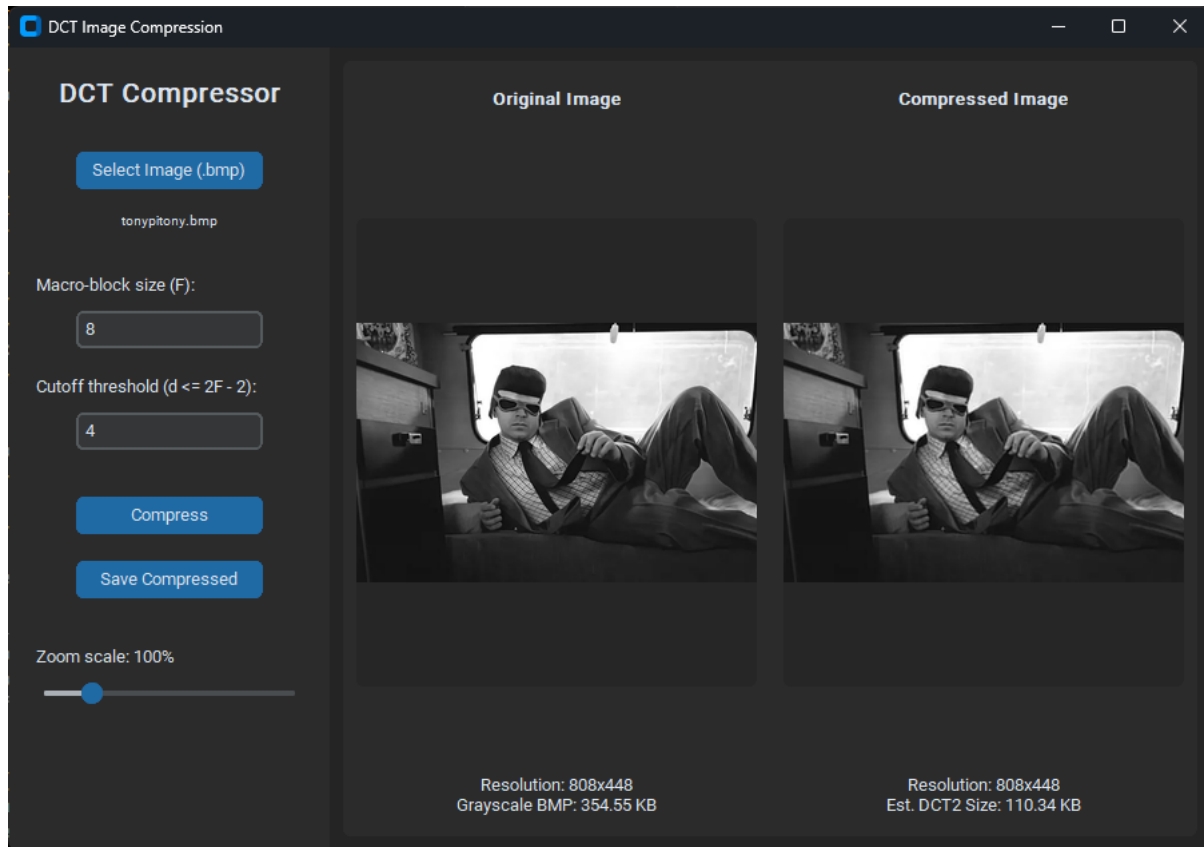


Figura 1.1: Interfaccia Grafica

1.4 Approfondimento: `scipy.fftpack`

La libreria `scipy.fftpack` si interfaccia internamente con algoritmi ottimizzati scritti in Fortran per il calcolo della trasformata di Fourier e delle sue versioni reali. L'implementazione efficiente della Trasformata Coseno Discreta (DCT) evita il calcolo matriciale diretto di tipo $\mathcal{O}(N^2)$ avvalendosi dell'algoritmo Fast Fourier Transform (FFT).

In termini teorici, il segnale di input di lunghezza N viene esteso e simmetrizzato ai bordi per generare un segnale artificiale periodico di lunghezza $2N$. Su tale array esteso viene poi applicato un algoritmo FFT di tipo divide-et-impera. Sfruttando le proprietà di simmetria del segnale, l'implementazione è in grado di estrarre unicamente i coefficienti reali associati alle componenti cosenoidali, annullando le componenti immaginarie in fase di calcolo. Questo approccio algoritmico riduce notevolmente la complessità asintotica, rendendo l'operazione scalabile ed efficiente per l'elaborazione di immagini

1.4.1 Complessità Computazionale

Il vantaggio principale nell'utilizzo di `scipy.fftpack` risiede nella netta riduzione della complessità computazionale asintotica rispetto all'implementazione manuale. L'approc-

cio *fast* garantisce infatti prestazioni nettamente superiori rispetto al classico prodotto matriciale:

Custom DCT 1D	$\mathcal{O}(N^2)$	(Moltiplicazione matrice-vettore classica)
Custom DCT 2D	$\mathcal{O}(N^3)$	(Applicazione iterativa 1D per righe e colonne)
Fast DCT 2D	$\mathcal{O}(N^2 \log N)$	(Approccio divide-et-impera via FFT)

Questo salto di complessità rende l'implementazione basata su SciPy asintoticamente più veloce di un fattore proporzionale a $N/\log N$ rispetto alla controparte *custom*, giustificandone l'adozione per l'elaborazione di matrici ad alta dimensionalità.

1.4.2 Firma del Metodo e Parametri

Come specificato nella documentazione ufficiale, l'interfaccia API esposta per il calcolo della DCT monodimensionale presenta la seguente firma:

```
scipy.fftpack.dct(x, type=2, n=None, axis=-1, norm=None, overwrite_x=False)
```

Di seguito si riporta la descrizione dei parametri ammessi dalla funzione e il relativo valore di ritorno:

Parametri in Ingresso:

- **x** (*array_like*): Matrice o tensore di input contenente i valori nel dominio spaziale su cui applicare la trasformata
- **type** ($\{1, 2, 3, 4\}$, *opzionale*): Specifica l'indice matematico della variante della DCT da calcolare. Il valore predefinito è 2, corrispondente alla DCT-II, formulazione standard universalmente impiegata nei protocolli di compressione dell'immagine
- **n** (*int*, *opzionale*): Definisce la dimensione di troncamento o riempimento della trasformata
 - Qualora $n < x.shape[axis]$, la sequenza di input subisce un troncamento forzato
 - Qualora $n > x.shape[axis]$, la sequenza di input viene estesa tramite apposizione di zeri (zero-padding)
 - In assenza di specificazione, il default implica $n = x.shape[axis]$.
- **axis** (*int*, *opzionale*): Specifica la dimensione (l'asse dell'array) lungo la quale valutare l'algoritmo. Il valore predefinito è -1
- **norm** ($\{None, 'ortho'\}$, *opzionale*): Parametro dedicato al fattore di scala. Il default è `None`. Impostando il parametro `'ortho'`, si garantisce l'ortonormalità della trasformata: il risultato preserva l'energia del segnale (isometria spaziale-frequenziale), requisito fondamentale per garantire l'esattezza numerica in fase di ricostruzione invertita
- **overwrite_x** (*bool*, *opzionale*): Se abilitato (`True`), autorizza la libreria a invalidare l'array in ingresso durante il calcolo dei coefficienti, favorendo un impiego di memoria in-place. Il valore predefinito è `False`.

Valore Restituito:

- y (*ndarray di reali*): Matrice multidimensionale contenente i coefficienti risultanti dall'applicazione dell'operatore lineare sull'array originale

1.4.3 Integrazione nel Codice Sviluppato

Per calcolare i coefficienti sul piano bidimensionale (dominio spaziale $F \times F$), una prima iterazione viene applicata sui vettori colonna e il relativo output viene trasformato lungo i vettori riga.

Nel modulo `dct.py`, la funzione è implementata come segue:

dct.py (Fast DCT 2D)

```
def fast_dct2(f: np.ndarray) -> np.ndarray:
    return scipy.fftpack.dct(
        scipy.fftpack.dct(f, axis=0, norm='ortho'),
        axis=1, norm='ortho'
    )
```


Capitolo 2

Metodi DCT e Compressione

Le funzioni fondamentali per il calcolo della trasformata e dell'algoritmo di compressione sono implementate in Python 3 all'interno del modulo `dct.py`. Il codice è stato progettato per ridurre al minimo le dipendenze esterne, appoggiandosi esclusivamente a `numpy` per le operazioni di algebra lineare e a `scipy.fftpack` per l'applicazione FFT

2.1 Trasformata Coseno Discreta (DCT)

La Trasformata Coseno Discreta è una trasformazione lineare che converte un segnale dal dominio spaziale al dominio delle frequenze, rappresentandolo come una combinazione lineare di funzioni cosenoidali a diverse frequenze. Nell'ambito della compressione di immagini, la DCT gioca un ruolo cruciale grazie alla sua eccellente capacità di compattazione dell'energia: la maggior parte delle informazioni visivamente significative viene concentrata nei coefficienti a bassa frequenza, permettendo di scartare le alte frequenze con un impatto visivo trascurabile per l'occhio umano

2.1.1 Definizione Matematica

Per un vettore unidimensionale $f \in \mathbb{R}^N$, la variante standard DCT-II (Discrete Cosine Transform Type II) è definita dalla seguente equazione:

$$C_k = \alpha_k \sum_{n=0}^{N-1} f_n \cos\left(\frac{\pi k(2n+1)}{2N}\right), \quad k = 0, 1, \dots, N-1 \quad (2.1)$$

dove il fattore di scala α_k , necessario per garantire l'ortonormalità della base, è definito come:

$$\alpha_k = \begin{cases} \frac{1}{\sqrt{N}} & \text{se } k = 0 \\ \sqrt{\frac{2}{N}} & \text{se } k > 0 \end{cases} \quad (2.2)$$

Operando in ambito discreto, la trasformata è rappresentabile in forma matriciale come operazione lineare:

$$\mathbf{C} = \mathbf{D}\mathbf{f} \quad (2.3)$$

dove $\mathbf{D}$ rappresenta la matrice DCT ortonormale di dimensione $N \times N$.

2.1.2 DCT 1D: Implementazione Manuale

L'implementazione manuale della trasformata monodimensionale si traduce nel diretto calcolo del prodotto matrice-vettore:

$$\mathbf{C} = \mathbf{D}\mathbf{f} \quad (2.4)$$

Complessità Computazionale: $\mathcal{O}(N^2)$

Funzione: `custom_dct1(f)`

- Calcola la matrice $\mathbf{D}$ tramite la funzione d'appoggio `compute_D(N)`.
- Esegue il prodotto matriciale $\mathbf{D} \cdot \mathbf{f}$.
- Restituisce il vettore contenente i coefficienti trasformati.

Di seguito il codice Python delle due funzioni:

```
dct.py

import numpy as np

def compute_D(N: int) -> np.ndarray:
    """Computes the N x N DCT matrix (DCT-II, orthonormal)."""
    alpha = np.ones((N, 1)) * np.sqrt(2 / N)
    alpha[0, 0] = 1 / np.sqrt(N)
    i = np.arange(N)
    k = np.arange(N).reshape(-1, 1)
    return alpha * np.cos(k * np.pi * (2 * i + 1) / (2 * N))

def custom_dct1(f: np.ndarray) -> np.ndarray:
    """1D DCT via matrix multiplication."""
    return compute_D(len(f)) @ f
```

Per ottimizzare i tempi di esecuzione dell'algoritmo custom, la matrice $\mathbf{D}$ viene calcolata sfruttando la vettorizzazione di NumPy, evitando cicli for espliciti e garantendo la massima efficienza possibile per questa classe di complessità.

2.1.3 DCT 2D: Implementazione Manuale

Per estendere l'operatore al dominio bidimensionale (immagini), si sfrutta la proprietà di separabilità della DCT, applicando la trasformata monodimensionale prima lungo le colonne e successivamente lungo le righe della matrice di input:

$$\mathbf{C} = \mathbf{D}\mathbf{F}\mathbf{D}^T \quad (2.5)$$

dove $\mathbf{F}$ è la matrice immagine di dimensione $N \times N$ e $\mathbf{D}$ è la matrice di trasformazione descritta in precedenza. L'implementazione algoritmica segue questi passi:

1. Per ogni colonna j : $c_j = \mathbf{D}f_j$
2. Per ogni riga i del risultato intermedio: $c_i = \mathbf{D}c_i$

Complessità Computazionale: $\mathcal{O}(N^3)$ (poiché richiede due serie da N iterazioni di trasformate 1D, ciascuna dal costo di $\mathcal{O}(N^2)$).

Funzione: `custom_dct2(f)`

dct.py

```
def custom_dct2(f: np.ndarray) -> np.ndarray:
    """
    2D DCT: prima sulle colonne, poi sulle righe.
    Corrisponde a  $C = D * F * D^T$ .
    """
    N = f.shape[0]
    D = compute_D(N)
    c = np.zeros_like(f, dtype=float)

    for j in range(N):          # DCT su ogni colonna
        c[:, j] = D @ f[:, j]

    for i in range(N):          # DCT su ogni riga del risultato intermedio
        c[i, :] = D @ c[i, :]

    return c
```

2.1.4 DCT 2D: Implementazione Veloce

Per le applicazioni pratiche di elaborazione e compressione, l'approccio manuale risulta troppo oneroso in termini di tempi di esecuzione. Si ricorre pertanto all'implementazione veloce tramite `scipy.fftpack`, basata su algoritmi di Fast Fourier Transform (FFT).

Complessità Computazionale: $\mathcal{O}(N^2 \log N)$

Funzione: `fast_dct2(f)`

- Applica in sequenza due DCT 1D veloci (asse 0 e asse 1).
- Utilizza internamente paradigmi *divide-et-impera* ottimizzati in Fortran.
- Imponendo il parametro `norm='ortho'`, mantiene inalterata l'ortonormalità della base, assicurando un'equivalenza numerica rispetto all'implementazione manuale soggetta solo all'errore di macchina ($\| \text{custom} - \text{fast} \| < 10^{-10}$).

dct.py

```
import scipy.fftpack

def fast_dct2(f: np.ndarray) -> np.ndarray:
    """Fast 2D DCT using scipy.fftpack.  $\mathcal{O}(N^2 \log N)$ ."""
    return scipy.fftpack.dct(
        scipy.fftpack.dct(f, axis=0, norm='ortho'),
        axis=1, norm='ortho'
    )
```

2.2 Trasformata Inversa (IDCT)

Per compiere l'operazione di sintesi e ricostruire l'immagine dal dominio delle frequenze al dominio spaziale, si ricorre alla trasformata coseno discreta inversa:

$$f_n = \sum_{k=0}^{N-1} \alpha_k C_k \cos\left(\frac{\pi k(2n+1)}{2N}\right) \quad (2.6)$$

All'interno del progetto, tale operazione è delegata alla funzione `fast_idct2`. Il rispetto del vincolo di ortonormalità in entrambe le direzioni garantisce analiticamente che, in assenza di troncamenti selettivi delle frequenze, il segnale ricostruito sia perfettamente identico all'originale: $\text{IDCT2}(\text{DCT2}(\mathbf{F})) = \mathbf{F}$.

dct.py

```
def fast_idct2(c: np.ndarray) -> np.ndarray:
    """Fast 2D IDCT using scipy.fftpack."""
    return scipy.fftpack.idct(
        scipy.fftpack.idct(c, axis=0, norm='ortho'),
        axis=1, norm='ortho'
    )
```

2.3 Algoritmo di Compressione

La logica di compressione sviluppata emula i principi dello standard JPEG. L'algoritmo segmenta l'immagine in blocchi quadrati disgiunti di dimensione $F \times F$, calcola la mappa delle frequenze tramite DCT, impone a zero i coefficienti associati ai dettagli fini e, infine, applica la trasformata inversa per ricomporre il mosaico spaziale. L'intero processo si snoda nelle seguenti fasi:

1. **Caricamento:** L'immagine originale (.bmp) viene convertita in scala di grigi e mappata in un array bidimensionale in virgola mobile.
2. **Troncamento:** Per garantire che l'immagine sia divisibile esattamente per la dimensione del blocco F , le dimensioni (H, W) vengono adattate scartando gli eventuali pixel marginali:

$$H_{\text{trunc}} = \left\lfloor \frac{H}{F} \right\rfloor \cdot F, \quad W_{\text{trunc}} = \left\lfloor \frac{W}{F} \right\rfloor \cdot F \quad (2.7)$$

3. **Iterazione:** Per ogni blocco $F \times F$ individuato alle coordinate (i, j) :
 - (a) **Estrazione:** $\mathbf{B} = \text{img}[i : i + F, j : j + F]$
 - (b) **Trasformazione:** $\hat{\mathbf{B}} = \text{DCT2}(\mathbf{B})$
 - (c) **Mascheratura:** Creazione di una maschera binaria $\mathbf{M}$ tale che $\mathbf{M}[k, l] = 1$ se $(k + l) \geq d$, altrimenti 0.
 - (d) **Filtraggio (Sogliatura):** Azzeramento dei coefficienti specifici: $\hat{\mathbf{B}}[\mathbf{M}] = 0$.
 - (e) **Ricostruzione:** $\mathbf{B}' = \text{IDCT2}(\hat{\mathbf{B}})$

(f) **Aggiornamento:** Inserimento del blocco ricostruito nell'immagine finale
 $\text{img_comp}[i : i + F, j : j + F] = \mathbf{B}'$.

4. **Post-Elaborazione:** I valori continui calcolati dalla IDCT vengono arrotondati all'intero più vicino e saturati (*clipping*) per rientrare nel canonico range cromatico a 8 bit $[0, 255]$:

$$\text{img_final} = \text{uint8}(\text{clip}(\text{round}(\text{img_comp}), 0, 255)) \quad (2.8)$$

5. **Salvataggio:** Conversione dell'array numpy in un oggetto immagine PIL per il successivo salvataggio o rendering a schermo.

2.3.1 Analisi del Criterio di Sogliatura

Il cuore della compressione consiste nel discriminante utilizzato per azzerare i coefficienti, basato sulla somma degli indici di frequenza spaziale:

$$\text{Azzerare il coefficiente } \hat{\mathbf{B}}[k, l] \quad \text{se} \quad (k + l) \geq d \quad (2.9)$$

Poiché all'interno della matrice DCT le frequenze più basse si concentrano nell'angolo in alto a sinistra (dove k e l sono prossimi allo 0), questa disequazione genera un taglio triangolare che preserva l'energia macroscopica del blocco, scartando i dettagli ad alta frequenza situati in basso a destra.

Il parametro d è responsabile della qualità dell'immagine risultante:

- **Valori bassi di d :** Rendono la disuguaglianza facile da soddisfare, azzerando una massiccia porzione della matrice. Questo comporta un alto rateo di compressione ma una marcata perdita di fedeltà (comparsa di artefatti a blocchi)
- **Valori alti di d :** Fissando la soglia al limite massimo richiesto ($d = 2F - 2$), l'algoritmo in realtà scarta un solo coefficiente per blocco: quello all'angolo in basso a destra (corrispondente agli indici massimi $k = F - 1$ ed $l = F - 1$). Di conseguenza, per non avere alcuna perdita di informazione (compressione nulla), la soglia dovrebbe teoricamente essere alzata a $d = 2F - 1$. Questi valori alti producono immagini visivamente indistinguibili dall'originale, annullando però l'efficienza della compressione

Capitolo 3

Risultati

3.1 Test sulle Matrici

La validazione dell'implementazione DCT è stata condotta confrontando i risultati ottenuti con i valori attesi forniti dalle specifiche dell'assignment, utilizzando come riferimento una matrice di test 8×8 .

3.1.1 Test Matrice 1D

Sulla prima riga della matrice di test è stata applicata la trasformata DCT 1D manuale. Il risultato è stato confrontato con i valori teorici calcolando l'errore relativo tramite la norma euclidea:

$$\text{Errore Relativo} = \frac{\|\text{Risultato} - \text{Atteso}\|_2}{\|\text{Atteso}\|_2} \quad (3.1)$$

Vettore di Input:

$$\mathbf{f} = [231, 32, 233, 161, 24, 71, 140, 245] \quad (3.2)$$

Valori Attesi:

$$\text{DCT1}(\mathbf{f}) = [4.01 \times 10^2, 6.60, 1.09 \times 10^2, -1.12 \times 10^2, 6.54 \times 10^1, 1.21 \times 10^2, 1.16 \times 10^2, 2.88 \times 10^1]^T \quad (3.3)$$

Risultato Ottenuto:

$$[401.99, 6.60, 109.17, -112.79, 65.41, 121.83, 116.66, 28.80]^T \quad (3.4)$$

L'errore relativo è pari a $\varepsilon = 0.354652\%$.

Conclusione

L'implementazione 1D manuale corrisponde con elevata fedeltà ai risultati attesi. L'errore relativo dello 0.35% rientra ampiamente nella soglia di tolleranza massima consentita del 5%. Questo conferma la correttezza formale dello scaling ortogonale applicato alla trasformata

3.1.2 Test Matrice 2D

Sulla matrice 8×8 completa sono state applicate e confrontate due varianti algoritmiche:

1. **DCT 2D Manuale** (`custom_dct2`): Implementazione iterativa con complessità $\mathcal{O}(N^3)$.
2. **DCT 2D Veloce** (`fast_dct2`): Implementazione ottimizzata tramite `scipy.fftpack` con complessità $\mathcal{O}(N^2 \log N)$.

Matrice di Input

$$\begin{bmatrix} 231 & 32 & 233 & 161 & 24 & 71 & 140 & 245 \\ 247 & 40 & 248 & 245 & 124 & 204 & 36 & 107 \\ 234 & 202 & 245 & 167 & 9 & 217 & 239 & 173 \\ 193 & 190 & 100 & 167 & 43 & 180 & 8 & 70 \\ 11 & 24 & 210 & 177 & 81 & 243 & 8 & 112 \\ 97 & 195 & 203 & 47 & 125 & 114 & 165 & 181 \\ 193 & 70 & 174 & 167 & 41 & 30 & 127 & 245 \\ 87 & 149 & 57 & 192 & 65 & 129 & 178 & 228 \end{bmatrix} \quad (3.5)$$

I risultati del test sono riassunti nella Tabella 3.1:

Implementazione	Errore Relativo	Errore Massimo	Ortonormalità
Custom DCT 2D ($\mathcal{O}(N^3)$)	0.703051%	8.75	Verificata
Fast DCT 2D SciPy ($\mathcal{O}(N^2 \log N)$)	0.703051%	8.75	Verificata

Tabella 3.1: Validazione DCT 2D

Confronto Matrici

Matrice Attesa:							
1.11e3	4.40e1	7.59e1	-1.38e2	3.50	1.22e2	1.95e2	-1.01e2
7.71e1	1.14e2	-2.18e1	4.13e1	8.77	9.90e1	1.38e2	1.09e1
4.48e1	-6.27e1	1.11e2	-7.63e1	1.24e2	9.55e1	-3.98e1	5.85e1
-6.99e1	-4.02e1	-2.34e1	-7.67e1	2.66e1	-3.68e1	6.61e1	1.25e2
-1.09e2	-4.33e1	-5.55e1	8.17	3.02e1	-2.86e1	2.44	-9.41e1
-5.38	5.66e1	1.73e2	-3.54e1	3.23e1	3.34e1	-5.81e1	1.90e1
7.88e1	-6.45e1	1.18e2	-1.50e1	-1.37e2	-3.06e1	-1.05e2	3.98e1
1.97e1	-7.81e1	9.72e-1	-7.23e1	-2.15e1	8.13e1	6.37e1	5.90
Matrice Ottenuta:							
1.119e3	4.402e1	7.592e1	-1.386e2	3.50	1.221e2	1.950e2	-1.016e2
7.719e1	1.149e2	-2.180e1	4.136e1	8.777	9.908e1	1.382e2	1.091e1
4.484e1	-6.275e1	1.116e2	-7.638e1	1.244e2	9.560e1	-3.983e1	5.852e1
-6.998e1	-4.024e1	-2.350e1	-7.673e1	2.665e1	-3.683e1	6.619e1	1.254e2
-1.090e2	-4.334e1	-5.554e1	8.173	3.025e1	-2.866e1	2.441	-9.414e1
-5.388	5.663e1	1.730e2	-3.542e1	3.239e1	3.346e1	-5.812e1	1.902e1
7.884e1	-6.459e1	1.187e2	-1.509e1	-1.373e2	-3.062e1	-1.051e2	3.981e1
1.979e1	-7.818e1	9.723e-1	-7.235e1	-2.158e1	8.130e1	6.371e1	5.906

Figura 3.1: Confronto DCT 2D

Proprietà

- **Consistenza:** Sia la DCT Manuale che la Fast DCT producono esiti numericamente identici (scostamento massimo di 8.14×10^{-13}), confermando un'assoluta equivalenza algoritmica
- **Ortonormalità:** La proprietà $IDCT2(DCT2(\mathbf{x})) = \mathbf{x}$ è rispettata con un errore di ricostruzione pari a 5.27×10^{-16} , dovuto all'errore macchina per precisione float64
- **Precisione:** L'errore relativo è all'incirca del 0.70%, ampiamente all'interno della fascia di tolleranza richiesta

3.2 Benchmark

3.2.1 Configurazione del Test

Per verificare la complessità asintotica teorica ($\mathcal{O}(N^3)$ contro $\mathcal{O}(N^2 \log N)$), è stato progettato un benchmark comparativo con le seguenti specifiche:

- **Dimensioni (N):** Sequenza di test per $N \in \{16, 32, 64, 128, 256, 512, 1024\}$
- **Dati di Input:** Matrici pseudo-casuali contenenti valori reali uniformemente distribuiti in $[0, 1)$
- **Implementazioni:** Custom DCT (manuale) contro Fast DCT (SciPy)

3.2.2 Risultati

I tempi di esecuzione misurati sono riportati nella Tabella 3.2 e illustrati graficamente in Figura 3.2.

Dimensione (N)	Custom DCT (ms)	Fast DCT (ms)	Vantaggio (Rapporto)
16	0.161	0.220	0.73x
32	0.186	0.048	3.90x
64	0.266	0.043	6.12x
128	0.877	0.104	8.46x
256	3.666	0.575	6.37x
512	42.644	2.819	15.13x
1024	267.628	16.052	16.67x

Tabella 3.2: Tempi di Esecuzione

Analisi dei Dati

- $N \leq 32$: Per matrici di piccole dimensioni, i tempi di esecuzione sono comparabili (inferiori al millisecondo). In alcuni casi, l'overhead della libreria SciPy può rendere la versione manuale leggermente più rapida
- $N \geq 64$: Il divario cresce in modo proporzionale alla dimensione della matrice
- $N = 1024$: A questa risoluzione, la versione Fast impiega circa 16 ms, mentre la versione manuale richiede 267 ms

3.2.3 Conclusioni

Sebbene la Custom DCT rimanga accessibile per singoli blocchi isolati, risulta inadeguata per l'elaborazione di intere immagini. Per $N \geq 256$, l'adozione dell'algoritmo basato su FFT diventa un requisito pressoché obbligatorio per garantire prestazioni accettabili

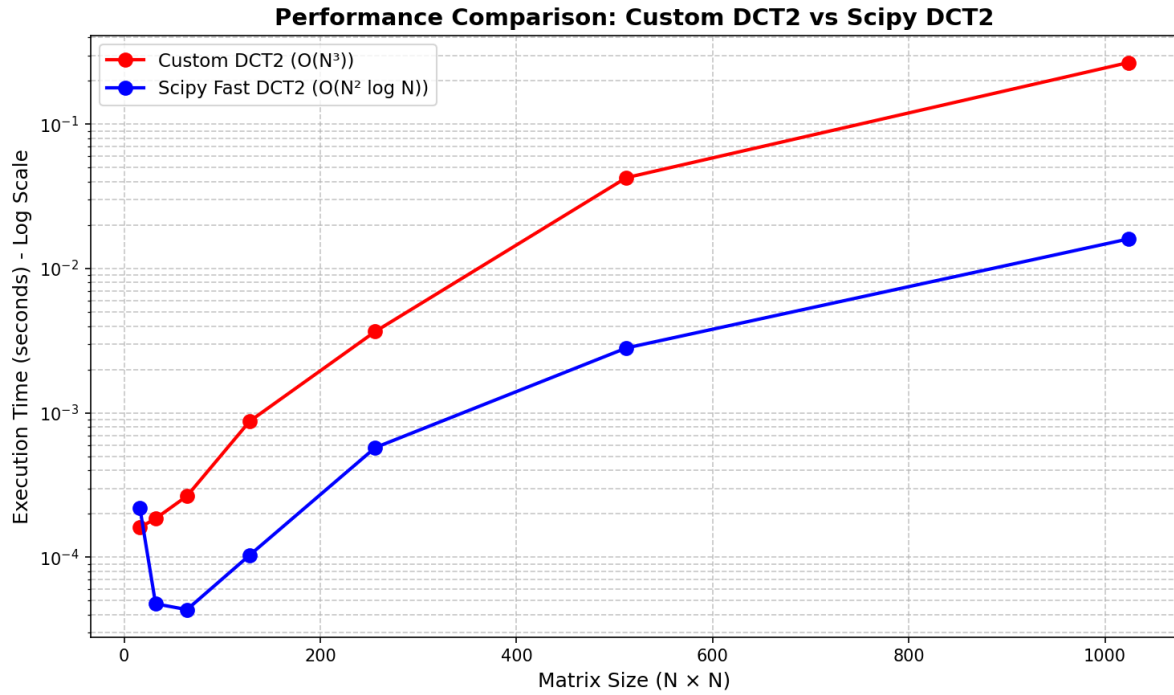


Figura 3.2: Custom DCT2 vs Fast DCT2

3.3 Compressione su Immagini

L'algoritmo a blocchi è stato testato su un set di immagini al fine di valutare il trade-off tra qualità visiva, tempo di calcolo e tasso di compressione al variare dei parametri.

3.3.1 Parametri e Configurazione

Il dataset di test comprende:

- **Immagini Artificiali:** `gradient.bmp`
- **Immagini ad Alta Risoluzione:** `bridge.bmp`, `cathedral.bmp`, `deer.bmp`
- **Immagini a Bassa e Media Risoluzione:** `shoe.bmp`, `tonypitony.bmp`

L'indagine si è concentrata sui due parametri fondamentali dell'algoritmo:

- **Dimensione Macro-blocco (F):** Regola la griglia di suddivisione (generalmente $F \in \{4, 8, 16\}$).
- **Soglia di Cutoff (d):** Determina quanti coefficienti frequenziali preservare ($0 \leq d \leq 2F - 2$).

3.3.2 Risultati

La Tabella 3.3 illustra i tempi di compressione misurati su tre immagini campione

Immagine	F	d	Tempo (ms)
gradient.bmp (1000×600)	8	4	129.4
	8	8	129.8
	8	12	139.1
	8	14	136.9
bridge.bmp (2749×4049)	8	4	4144.0
	8	8	4183.4
	8	12	4136.3
	16	16	2470.9
shoe.bmp (260×260)	8	4	17.0
	8	8	14.6
	8	12	15.5
	4	6	52.2

Tabella 3.3: Tempi di Compressione

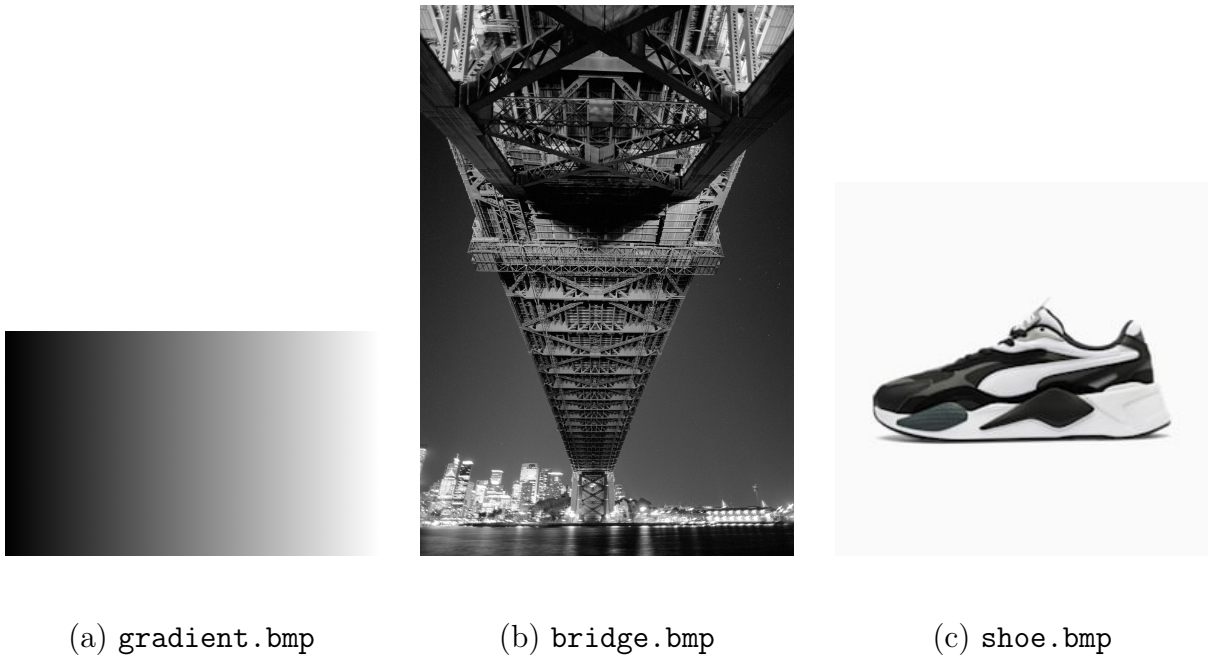


Figura 3.3: Campioni Utilizzati

3.3.3 Analisi dei Risultati

- gradient.bmp (1000×600):
 - Il tempo di elaborazione si attesta intorno ai 134 ms, manifestando una totale indipendenza dal parametro d
 - Il valore della soglia influisce esclusivamente sulla mappatura dei coefficienti da azzerare, ma non altera il numero di trasformate DCT calcolate dall'algoritmo
- bridge.bmp (2749×4049):

- Il processamento di circa 11 milioni di pixel costituisce carico computazionale non indifferente
 - Fissato $F = 8$, il tempo globale sale a circa 4.2 s (31 volte superiore al gradiente)
 - Ampliando il blocco a $F = 16$, il tempo scende a 2.5 s, poiché l'algoritmo deve istanziare e iterare su un numero minore di blocchi
- `shoe.bmp` (260×260):
 - Su risoluzioni molto contenute l'elaborazione è quasi istantanea
 - Interessante notare come, riducendo la dimensione a $F = 4$, il tempo triplichi (~ 54 ms) rispetto a $F = 8$: a parità di risoluzione totale, un blocco più piccolo impone un numero di invocazioni della DCT 4 volte superiore

3.3.4 Osservazioni

Per l'analisi visiva degli artefatti, è stata utilizzata l'immagine di test `tonypitony.bmp` (risoluzione 808×450).

Soglia di Cutoff (d)



(a) Originale (1065.35 KB su disco, RGB)



(b) $F = 8$, $d = 2$ (83.58 KB, PNG est.)



(c) $F = 8$, $d = 4$ (110.34 KB, PNG est.)



(d) $F = 8$, $d = 8$ (117.17 KB, PNG est.)

Figura 3.4: Differenza al variare di d

Per quantificare oggettivamente il degrado visivo, si è fatto ricorso alla metrica PSNR (Peak Signal-to-Noise Ratio), calcolata tramite il modulo di validazione. Valori di PSNR elevati (tipicamente sopra i 30 dB) indicano un'ottima fedeltà rispetto all'originale, mentre

valori inferiori riflettono la perdita di dettagli e la comparsa di artefatti. Analizzando l'immagine compressa `tonypitony.bmp` con $F = 8$, si rileva che per $d = 8$ il PSNR si mantiene su valori ampiamente soddisfacenti pari a $40.31dB$ (Figura 3.4d). Al contrario, abbassando la soglia a $d = 2$, il PSNR scende a $26.01dB$ (Figura 3.4b), confermando la perdita di informazione visiva percepibile a occhio nudo.

Dal punto di vista prestazionale, le tempistiche si mantengono stabili al variare di d (137.07 ms per $d = 2$ contro 138.73 ms per $d = 8$). L'operazione di mascheratura condotta sui coefficienti **non** influisce né sull'onere computazionale né sulla dimensione del file BMP salvato su disco (costante per definizione del formato: $W_{\text{trunc}} \times H_{\text{trunc}} + 1078$ byte). Essa determina esclusivamente la **qualità visiva** dell'immagine ricostruita e la sua **compressibilità strutturale**, quest'ultima riflessa nella stima PNG lossless mostrata dalla GUI.

Dimensione del Macro-blocco (F)



(a) Originale (1065.35 KB su disco, RGB)



(b) $F = 4, d = 3$ (112.47 KB, PNG est.)



(c) $F = 10, d = 3$ (97.10 KB, PNG est.)



(d) $F = 16, d = 3$ (81.36 KB, PNG est.)

Figura 3.5: Degrado all'aumentare di F

A differenza della soglia d , il parametro F influisce notevolmente sulle prestazioni temporali:

- $F = 4$: Richiede 480.51 ms, a causa dell'elevato numero di blocchi iterati
- $F = 10$: Scende a 92.90 ms
- $F = 16$: Ottimizza ulteriormente il tempo a 44.23 ms

3.3.5 Conclusioni Finali

I vari scenari valutati confermano quanto ipotizzato in teoria:

- Il range $F \in [8, 10]$ garantisce il compromesso visivo e prestazionale ideale, bilanciando il costo computazionale con la soppressione degli artefatti visivi (proprio per questo, $F = 8$ è lo standard del formato JPEG)
- L'impostazione della soglia di taglio d diviene, pertanto, l'unico vero fattore di regolazione che l'utente deve operare per bilanciare la **qualità visiva** dell'immagine contro la sua **compressibilità strutturale** (misurata tramite la stima PNG lossless mostrata dalla GUI), senza alcun effetto sulla dimensione del file BMP fisicamente salvato su disco.

3.4 Dimensione del File

3.4.1 Problema Ricontrato

Durante lo sviluppo dell'interfaccia grafica è emerso un comportamento apparentemente anomalo nella visualizzazione della dimensione stimata del file compresso. Nello specifico, si osservava che:

- La dimensione dell'immagine **originale** riportata nella GUI corrispondeva correttamente alla dimensione reale del file BMP su disco.
- La dimensione dell'**immagine compressa**, pur risultando **inferiore** a quella originale, rimaneva **pressoché invariata al variare dei parametri F e d** .

Tale comportamento era in contrasto con l'effetto atteso dell'algoritmo DCT2: al diminuire di d (o al crescere di F), vengono azzerati più coefficienti, si perde più informazione e l'immagine diventa più comprimibile.

3.4.2 Causa

L'anomalia è riconducibile a due fattori distinti e indipendenti tra loro.

Fattore 1 — Conversione (RGB $\rightarrow$ Scala di Grigi)

La funzione `compress_image` in `compressor.py` effettua la conversione in scala di grigi al momento del caricamento:

```
img_array = np.array(Image.open(image_path).convert('L'), dtype=float)
```

Qualora l'immagine BMP in input sia codificata in formato RGB a 24 bit (3 byte per pixel), l'immagine ricostruita dalla DCT è invece in modalità 'L' a 8 bit (1 byte per pixel). Il formato BMP non comprime i dati, pertanto la dimensione del file dipende esclusivamente dal prodotto:

$$\text{Dimensione BMP} = W \times H \times \text{byte per pixel} + \text{header} \quad (3.6)$$

Ne consegue che il rapporto di riduzione apparente tra dimensione originale (RGB, 3 byte/pixel) e dimensione compressa (grayscale, 1 byte/pixel) si attesta sistematicamente a circa $\frac{1}{3}$, indipendentemente dai parametri F e d . Tale riduzione è dunque un artefatto della conversione del modo colore, non un effetto della compressione DCT2.

Nota: la specifica del progetto richiede esplicitamente immagini in input già in toni di grigio. Per input conformi (BMP grayscale a 8 bit), la conversione `.convert('L')` è un'operazione nulla e il Fattore 1 non introduce alcuna distorsione nelle dimensioni. Il fattore rimane tuttavia rilevante come misura difensiva per garantire la correttezza del processing anche in presenza di file BMP non conformi (es. codificati in RGB).

Fattore 2 — Formato BMP (Uncompressed)

Il formato BMP è per definizione privo di compressione dei dati: ogni pixel occupa sempre il medesimo numero di byte, indipendentemente dal suo valore numerico. Pertanto, anche a parità di modo colore, azzerare i coefficienti DCT modifica i valori dei pixel ma non il numero di byte scritti sul file. Questo implica che:

$$\forall F, d: \quad \text{Dimensione BMP compressa} = W_{\text{trunc}} \times H_{\text{trunc}} \times 1 \text{ byte} + \text{costante} \quad (3.7)$$

Di conseguenza, il parametro `quality=100` passato a PIL in fase di salvataggio BMP era privo di effetto: tale argomento è semanticamente valido solo per formati lossy come JPEG, e viene ignorato dal writer BMP della libreria Pillow.

3.4.3 Soluzione Adottata

Per ottenere una stima della dimensione che rifletta esclusivamente l'effetto della compressione DCT2, si è adottata la seguente strategia:

1. **Eliminazione del parametro `quality`:** il salvataggio su disco del file BMP compresso avviene ora con la sola chiamata `save(filename, format="BMP")`, senza argomenti aggiuntivi non validi per tale formato.
2. **Dimensione originale tramite formula analitica:** la dimensione dell'immagine originale (convertita in grayscale e troncata) è calcolata direttamente come:

$$\text{Dimensione BMP} = W_{\text{trunc}} \times H_{\text{trunc}} + 1078 \quad [\text{byte}] \quad (3.8)$$

dove $1078 = 14 + 40 + 256 \times 4$ byte corrisponde all'header fisso di un BMP a 8 bit con palette (file header + DIB header + palette a 256 colori). Questo valore coincide esattamente con la dimensione del file BMP scritto da PIL su disco, senza necessità di alcuna codifica.

3. **Stima compressa tramite PNG lossless:** la dimensione dell'immagine compressa è calcolata codificando il risultato della DCT2 in un buffer PNG in memoria:

```
buf = io.BytesIO()
comp_img.save(buf, format="PNG", optimize=True)
size_kb = len(buf.getvalue()) / 1024
```

Il formato PNG è lossless (non introduce distorsioni aggiuntive rispetto alla DCT2) e sfrutta la ridondanza strutturale: i blocchi piatti prodotti dall'azzeramento delle frequenze vengono compressi molto più efficacemente rispetto ai blocchi originali ad alta variabilità. La stima varia quindi in modo coerente con F e d .

4. **Comportamento a due fasi della GUI:** è opportuno precisare che la label relativa alla dimensione dell'immagine originale si aggiorna in due momenti distinti:

- **Fase di selezione** (`select_image`): mostra l'etichetta "File Size (su disco)" calcolata tramite `os.path.getsize()`, ossia la dimensione reale del file BMP su disco (che può essere RGB o grayscale a seconda dell'input).
- **Fase post-compressione** (`process_image`): l'etichetta si aggiorna in "Grayscale BMP" e mostra la dimensione analitica $W_{\text{trunc}} \times H_{\text{trunc}} + 1078$ byte, coerente con la modalità 'L' dell'immagine elaborata.

Per input conformi alla specifica (BMP già in scala di grigi a 8 bit), le due misure coincidono a meno del troncamento geometrico.

3.4.4 Validazione della Soluzione

La Tabella 3.4 riporta le dimensioni misurate per l'immagine `tonypitony.bmp` (risoluzione originale 808×450 , troncata ai multipli di F : ad esempio 808×448 per $F = 8$) al variare di F e d . La colonna Grayscale BMP è calcolata analiticamente come $W_{\text{trunc}} \times H_{\text{trunc}} + 1078$ byte (header fisso del formato BMP a 8 bit con palette); la colonna DCT2 (PNG) corrisponde alla dimensione del buffer PNG lossless ottenuto codificando l'immagine ricostruita in memoria.

F	d	Grayscale BMP (KB)	DCT2 PNG est. (KB)
8	2	354.55	83.58
8	4	354.55	110.34
8	8	354.55	117.17
4	3	354.55	112.47
10	3	352.62	97.10
16	3	351.05	81.36

Tabella 3.4: Dimensioni al variare di F e d (`tonypitony.bmp`)

I dati confermano il comportamento atteso:

- **Al crescere di d** (fissato $F = 8$): la stima PNG aumenta da 83.58 KB ($d = 2$) a 117.17 KB ($d = 8$), poiché più coefficienti vengono preservati, l'immagine ricostruita è più ricca di dettagli e quindi meno comprimibile da un codec lossless.
- **Al crescere di F** (fissato $d = 3$): la stima PNG diminuisce da 112.47 KB ($F = 4$) a 81.36 KB ($F = 16$), perché blocchi più grandi consentono di azzerare una frazione maggiore di coefficienti per blocco, producendo immagini più uniformi e strutturalmente ridondanti.

- **La colonna Grayscale BMP** è quasi costante (varia solo per il troncamento geometrico al variare di F) e corrisponde esattamente alla dimensione del file che PIL scrive su disco in formato BMP, confermando la correttezza della formula analitica adottata.

Capitolo 4

Conclusioni

Questo progetto ha consentito di esplorare il ruolo della Trasformata Coseno Discreta nell'ambito della compressione delle immagini digitali, replicando i principi fondamentali dello standard JPEG.

Sintesi dei Risultati

- **Implementazione:** Entrambe le varianti della DCT — manuale (`custom_dct2`) e veloce (`fast_dct2`) — hanno superato i test di validazione con un errore relativo inferiore all'1%, ben al di sotto della soglia di tolleranza del 5% indicata dalla specifica. La proprietà di ortonormalità $\text{IDCT2}(\text{DCT2}(\mathbf{x})) = \mathbf{x}$ è stata verificata con un errore di ricostruzione dell'ordine dell'errore macchina ($\sim 10^{-16}$), confermando la validità dell'implementazione
- **Prestazioni:** Il benchmark ha confermato le previsioni teoriche di complessità asintotica. La DCT manuale scala come $\mathcal{O}(N^3)$, diventando proibitiva già per $N > 512$, mentre la versione basata su `scipy.fftpack` mantiene un andamento $\mathcal{O}(N^2 \log N)$, risultando fino a ~ 16 volte più veloce a $N = 1024$. L'adozione della versione fast è pertanto un requisito necessario per l'elaborazione di immagini
- **Compressione:** Gli esperimenti su immagini reali hanno evidenziato il ruolo dei due parametri F e d :
 - La soglia d controlla direttamente la qualità visiva (PSNR): valori bassi producono artefatti a blocchi marcati (PSNR ≈ 26 dB), mentre valori alti preservano la fedeltà dell'immagine (PSNR ≈ 40 dB), senza alcun impatto sui tempi di elaborazione
 - La dimensione F del macro-blocco influisce invece sulle prestazioni temporali: blocchi più piccoli moltiplicano il numero di chiamate DCT, aumentando significativamente il tempo di calcolo. Il valore $F = 8$, standard del formato JPEG, offre il compromesso ottimale tra qualità e velocità